CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 8: Binary Tree Traversals and Binary Search Tree Insertion

Practical Aim:

To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree.

Problem 1 :

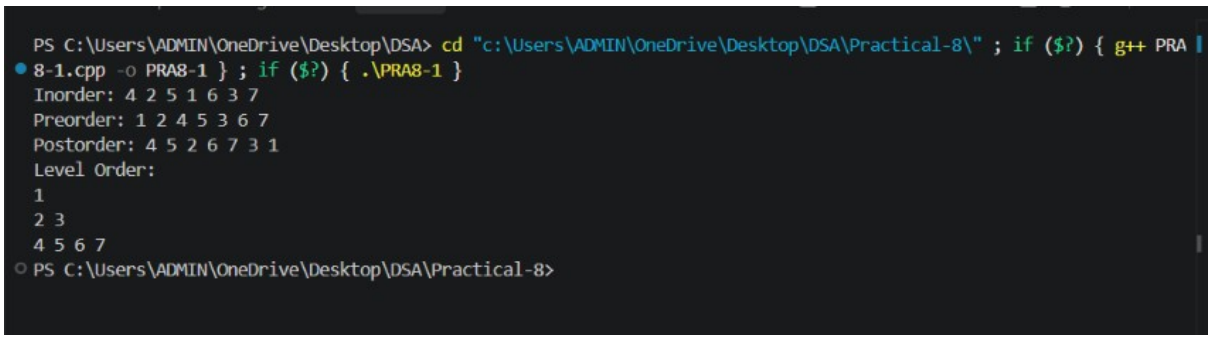
A company stores its organizational hierarchy as a tree — the CEO at the top, managers below, and employees at the leaves. Different departments need the hierarchy printed in different orders: HR wants each manager printed between their left and right sub-teams, the archive department wants each manager printed before their sub-teams, the payroll department wants each manager printed after all their sub-teams are listed, and the floor manager wants everyone printed level by level from top to bottom. Given a binary tree representing the hierarchy, implement all four printing orders and display the result of each.

Describe how your solution for level-by-level printing differs from the other three. What additional structure did you need to use, and why does the same recursive approach that works for the other three not directly apply here?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-8

Output:



Problem Faced:

The main challenge was keeping track of which traversal corresponds to which department's requirement — HR's request is an inorder traversal, the archive department's request is a preorder traversal, and payroll's request is a postorder traversal — and implementing the level-by-level (level order) traversal, which could not reuse the same simple recursive pattern as the other three and needed an auxiliary queue instead.

Outcome/Conclusion:

All four traversals were implemented on the sample hierarchy tree: inorder (left subtree, node, right subtree) for HR, preorder (node, left subtree, right subtree) for the archive department, postorder (left subtree, right subtree, node) for payroll, and level order for the floor manager, with each traversal's output printed separately.

Level-by-level printing differs from the other three because it must visit nodes breadth-first, i.e. all nodes at one depth before moving to the next depth, whereas inorder, preorder, and postorder are all depth-first and naturally expressed through simple recursive calls on the left and right children. The additional structure needed is a queue: the root is pushed first, and then repeatedly a node is popped from the front, printed, and its children are pushed to the back, which keeps nodes grouped level by level. The same plain recursive approach used for the other three does not directly apply here because recursion follows the call stack and goes fully down one branch before returning, which naturally produces a depth-first order, not the breadth-first order that level order printing requires.

Problem 2 :

A school library uses a system where every book is assigned a unique code when it arrives. Books are organized so that at any shelf position, all books to the left have smaller codes and all books to the right have larger codes. This arrangement lets staff find any book quickly by always knowing which direction to look. Given a sequence of book codes arriving one by one, insert each into the correct position and print the inorder sequence after all insertions are complete.

Describe how your solution decides where to place each new book code. What property of the final inorder sequence tells you whether all insertions were performed correctly?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-8

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-8\" ; if ($?) { g++ PRA
8-2.cpp -o PRA8-2 } ; if ($?) { .\PRA8-2 }
Inorder sequence: 20 30 40 50 60 70 80
PS C:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-8>
```

Problem Faced:

The main challenge was implementing the binary search tree insertion logic correctly: starting from the root and repeatedly moving left or right based on comparison with the current node, then creating the new node once an empty position (a NULL link) is reached, while being careful about how the very first insertion creates the root.

Outcome/Conclusion:

Binary search tree insertion was implemented by comparing each incoming book code with the code at the current node, starting at the root: if the new code is smaller, the search moves to the left child; if it is larger, it moves to the right child; this continues until an empty spot is found, where the new node is attached. After all book codes were inserted, an inorder traversal was performed and printed.

Each new book code decides its own position purely through comparisons against the nodes already present: it travels left whenever it is smaller than the current node and right whenever it is larger, which is exactly what keeps all smaller codes to the left and all larger codes to the right of every node. The

property that confirms correct insertion is that the inorder traversal of a correctly built binary search tree always comes out in strictly increasing (sorted) order; if the printed sequence is sorted, every insertion placed its code on the correct side at every step.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] Preorder traversal always visits the root before anything else. Using only this property — without tracing any tree — explain whether it would be possible to reconstruct the original tree from its preorder output alone, and what additional information you would need if it is not.

Sol: For an arbitrary binary tree, preorder output alone is not enough to reconstruct the tree, because preorder only tells you the root first, but it does not mark where the left subtree ends and the right subtree begins — the same preorder sequence could correspond to several different tree shapes. To reconstruct the tree uniquely, additional information is needed, typically the inorder traversal of the same tree (since inorder locates the root's position and splits the left and right subtrees), or explicit markers indicating missing (NULL) children at each step. However, if the tree is known to be a binary search tree, preorder alone is sufficient: since every value less than the root belongs to the left subtree and every value greater belongs to the right subtree, the split point can be found directly from the values themselves, without needing a second traversal.

[Problem 2] The inorder traversal of a correctly built BST always produces a sorted sequence. If your inorder output is not sorted after all insertions, what does that tell you about where the error is — in the insertion logic, the traversal logic, or possibly both?

Sol: If the inorder output is not sorted, it tells you that either the tree does not actually satisfy the BST property (an insertion placed a code on the wrong side at some node), or the traversal itself is not visiting left subtree, node, right subtree in the correct order, or possibly both at once. To isolate which one is at fault, the tree's structure can be checked independently of the traversal — for example, by printing the tree level by level (level order) and manually verifying that every node's left child is smaller and right child is larger — while

separately verifying the traversal logic on a tree whose structure is already known to be correct (such as a tree built by hand or hard-coded). If the structure check fails, the fault is in the insertion logic; if the structure is correct but the printed order is still wrong, the fault is in the traversal logic; if both checks fail, both parts need correction.